Resources in Kubernetes



How to give info to the Scheduleder about our workloads?

Info about workloads

Goals

Info for the Scheduler and for the Container Runtime (Docker)

- The scheduler **needs** to know the memory/CPU usage of workloads
- We provide this info with `requests` definitions
- You can use `limits` to automate handling of buggy processes

Info for ourselves

- Your colleagues and your future self will thank you for having this info
- Capacity planning/budgeting is an part of working with the cloud

Load testing

- Option 1: recorded traffic from real-world load and replay
- Option 2: some load testing framework, e.g. Locust

Let’s start with an example

```yaml
apiVersion: v1
kind: Pod
metadata: { name: frontend }
spec:
  containers:
  - name: app
    image: images.my-company.example/app:v4
    resources:
      requests:
        memory: 256Mi
        cpu: 250m
      limits:
        memory: 512Mi
        cpu: 750m
  - name: log-aggregator
    image: images.my-company.example/log-aggregator:v6
    resources:
      requests:
        memory: 128Mi
        cpu: 500m
      limits:
        memory: 512Mi
        cpu: 750m
```

- 2 containers in 1 pod
- We can limit/requests CPU and memory
- 1 CPU is 1 core
  (entry in `/proc/cpuinfo`)
- 1m CPU is 1/1000 of a core
- In newer versions of Kubernetes even disk space and disk IO
- And they are working on net bandwidth
- The **requests** are **added** and count **together** for the **whole Pod**
- The **limits** are considered **separately** for each **container**

`requests`

Semantics

- The **estimated** resource usage of the container
- Used for scheduling (to know how big is this pod)
- **Never overcommitted**, but **flexible**
- If no space, then the pod stays pending
- If CPU is **breached**, **possibility** of **throttling**
- If memory is **breached**, **possibility** of **eviction** (next slide!)

Evictions on memory `requests` breach

- Pods can be evicted (== deleted) by Kubernetes
- No evictions will happen on a node with free memory
- Evictions can happen if a pod uses more memory than requested
- Pods without requests will be evicted first, then pods in breach
- Question: if no pod uses more than requested, will there be evictions?

`limits`

Semantics

- If this is **breached**, then the program is **misbehaving**
- **Can be overcommitted**, but **rigid**
- Not considered for scheduling, only for runtime
- If memory, then **restart** the container (not the pod)
- If CPU, then **throttle** the container

Default values

What if only `requests` or `limits` are defined

Without `limits`, but with `requests`

- The specified `requests` will still work
- Docker gets no info: no container restarts will happen

Without `requests`, but with `limits`

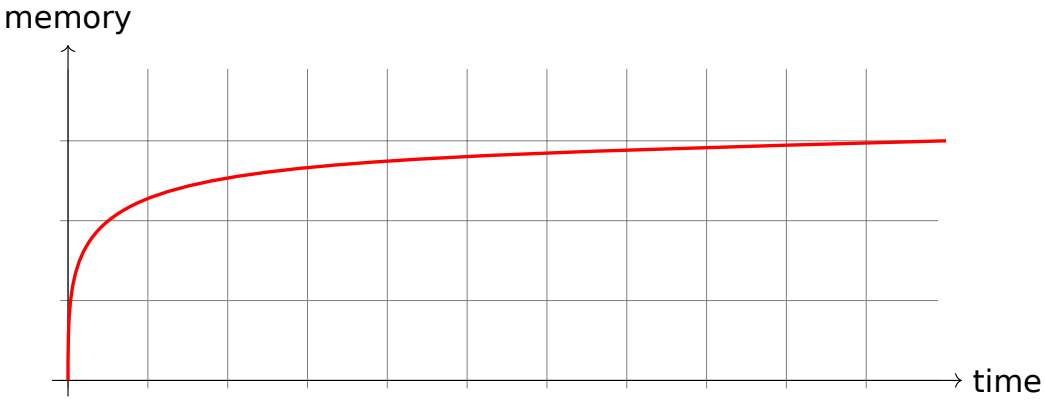
- The specified `limits` will be also used as `requests`
- In Kubernetes lingo these pods are called `Guaranteed`
- Because this pod is not likely to be evicted ever: it can't be in breach

Without anything

- `limit` defaults to unlimited, `request` defaults to 0
- Not production ready

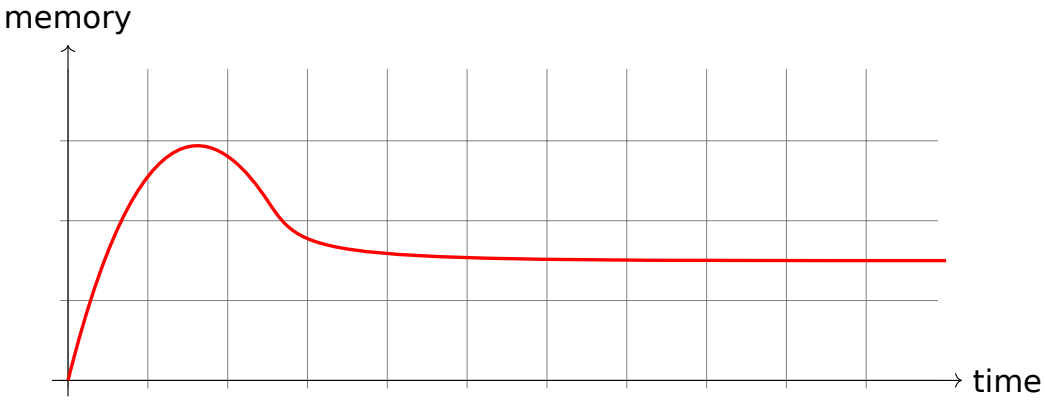
Requests and limits

How to set request and limit?



Requests and limits

How to set request and limit?



Thanks!